

HIP-009 — Arquitetura de Sincronização (Fase 2 · Etapa 3)

Status: Approved · **Architectural Baseline** · **Versão:** 1.0 · **Histórico:** v1.0 (2026-07-20) criação. Pré-requisito de QUALQUER código (inclusive /mobile/ingest). Sob ADR-000 · ARCH-002 · deriva de HIP-007. Decisão: ADR-008. **Escopo:** robusta para **toda aquisição observacional** (wearables, dispositivos médicos, sensores contínuos, apps, agregadores, FHIR, documentos) — não só wearables.

1. Princípios da sincronização

- 1. **SSOT bruto imutável + idempotente:** toda Observação é gravada crua com proveniência completa; ingestão é **idempotente** por chave determinística. **Nunca** sobrescreve a fonte; correções geram **nova versão**.
- 2. **Reconciliação na PROJEÇÃO, nunca no bruto:** origens concorrentes coexistem no bruto (traçabilidade); a escolha de um representante acontece na camada de projeção/indicador — **sem destruir origem**.
- 3. **Recompute self-healing:** projeções/indicadores são **recalculados** por chave (métrica × janela) quando chega dado novo/atrasado/corrigido → o sistema se auto-corrige sem edição destrutiva.
- 4. **API-first:** uma única API de ingestão serve o app (push) e conceitualmente qualquer cliente; conectores web (pull) convergem para a MESMA persistência canônica.

2. Quem inicia a sincronização (por classe)

Classe	Iniciador	Gatilhos
Mobile (push)	o app	ao abrir (foreground) · background (OS) · mudança de dado na saúde do aparelho
Web/Fabricante (pull)	o backend	on-open (throttle) · webhook · agendado
Agregador	o agregador	webhook/stream → mesma ingestão
O app é o iniciador primário do monitoramento contínuo; o backend nunca "puxa" do celular (não há API de nuvem).		

3. Sincronização em background + offline + idempotência (mobile)

- **Background:** iOS `BGTaskScheduler` + HealthKit background delivery; Android `WorkManager` + Health Connect. Sujeito a **limites do SO** (bateria/throttle) → estratégia **oportunista + catch-up ao abrir**.
- **Cursor incremental por (usuário × fonte × dispositivo):** cada origem tem sua marca d'água; o app envia só o delta.
- **Fila offline:** Observações não enviadas ficam em **fila local persistente** (perda de rede não perde dado); reenvio com **backoff**; como a ingestão é **idempotente**, reenviar é seguro (duplicatas colapsam).
- **Idempotência (chave determinística):** `(user, source, deviceId, metric, recordedAt[, externalId])`. O `externalId` da fonte, quando existe, é a âncora mais forte; senão, a tupla acima. Reenvios e sobreposição colapsam para 1 registro.

4. Identidade: fontes e dispositivos

- **Fonte (source)** = origem lógica (Withings, Apple Health, Health Connect, agregador...).
- **Dispositivo (deviceId / deviceModel)** = aparelho concreto dentro da fonte (ex.: 2 balanças; iPhone + Apple Watch).
- **Registro de dispositivos:** `(user, platform, deviceId)` — o app registra seus aparelhos; cursores são **por dispositivo**. Isso sustenta as respostas de §5–§8.

5. (Q-A) Múltiplos dispositivos E múltiplas fontes ao mesmo tempo

Cada par (**fonte × dispositivo**) tem **cursor independente** e escreve Observações com sua própria proveniência. O SSOT bruto **preserva todas**; nada compete no bruto. Um usuário com iPhone + Apple Watch + Withings + Health Connect gera várias trilhas paralelas, todas rastreáveis. A convergência (o que o usuário **vê**) é decidida na projeção (§6–§7).

6. (Q-B) Observações equivalentes de origens diferentes

Ex.: uma pesagem na balança Withings chega **pela nuvem Withings E pelo Apple Health** (o app espelhou). São **duas Observações do mesmo fato real por caminhos diferentes**.

- **No bruto:** as duas são mantidas, cada uma com sua origem (traçabilidade — nunca apagamos origem).
- **Chave de EQUIVALÊNCIA** (na projeção): agrupa Observações "do mesmo fato" por `metric + janela temporal (+ deviceId / impressão de valor quando disponível)`. O grupo é **1 medida real** com N origens.
- **Representante:** a projeção escolhe **uma** por grupo (política de precedência, §7); as demais permanecem no bruto como origens equivalentes vinculadas. Assim não há duplicidade na tela, sem perder que "veio de duas origens".

7. (Q-C) Confiabilidade / precedência entre fontes concorrentes

Cada Observação carrega **nível de qualidade/evidência** (§HIP-007: `manual` < `consumer_device` < `medical_device` < `clinically_validated`; `derived` à parte) + **confiança** (medido vs estimado). A escolha do representante segue uma **política de precedência determinística e configurável** (não hardcoded):

1. **maior nível de evidência** (ex.: dispositivo médico > wearable de consumo > manual);
2. empate → **maior confiança**;
3. empate → **origem mais direta** (fabricante direto > espelho via plataforma de SO > agregador);
4. empate → **mais recente / versão mais nova**. A política atua **só na projeção/indicador** — o bruto guarda tudo. Mudar a política **recalcula** sem perda (self-healing). *(Valores/pesos exatos ficam para a implementação; aqui fixamos a FORMA da política sobre os metadados.)*

8. (Q-D) Rastreabilidade por toda a vida do dado

- **SSOT append-only + versionado**: cada Observação é imutável; correção da fonte = **nova versão** com `supersedes` para a anterior (histórico preservado, nunca edição in-place). Proveniência gravada: `source`, `connectorClass`, `deviceId`, `connectorVersion`, `externalId`, `ingestedAt`, `evidenceTier`, `reliability`, `version`.
- **Linhagem ponta a ponta**: cada ponto de **projeção** referencia a Observação **representante**; cada **indicador** referencia o **conjunto** de Observações que o originaram. Logo, de qualquer valor/insight na tela é possível voltar até **origem, dispositivo, versão e instante** exatos. Cumpre `princípio_rastreabilidade_documental` no plano observacional e alimenta a auditabilidade da inteligência (V3).

9. Resolução de conflitos, versionamento e dados atrasados

- **Bruto**: sem conflito (append-only + idempotente).
- **Correções da fonte**: nova versão supersede a anterior; projeção usa a última não-superseda.
- **Dados atrasados/retroativos**: como a projeção é por `métrica × janela`, uma Observação atrasada **dispara recompute** daquela janela → auto-cura, sem tocar o restante.
- **Conflito na projeção**: resolvido pela política de precedência (§7), determinístico e reprodutível.

10. Conectividade / perda de rede

Fila offline persistente no app + reenvio idempotente com backoff; o cursor só avança após confirmação do backend (at-least-once + idempotência = efetivamente exactly-once no SSOT).

11. Robustez universal (não só wearables)

Todos os mecanismos acima são **agnósticos à origem**: um CGM, um MAPA, um app parceiro, um bundle FHIR ou uma Observação extraída de documento entram pela mesma ingestão, com a mesma idempotência, equivalência, precedência e rastreabilidade. Adicionar origem = novo adaptador emitindo Observação — **zero mudança na arquitetura de sincronização**.

12. Contrato de ingestão (visão — implementação na Etapa 4)

Uma API única de ingestão recebe **lotes de Observações** com metadados completos + chave de idempotência; valida, grava o SSOT bruto (idempotente/versionado) e agenda o recompute das projeções/NOV-001. Usada pelo app (push) e convergente com o pull dos conectores web. **Detalhes e código só na Etapa 4.**

13. Decisões / pendências (após aprovação)

- Confirmar a **forma da política de precedência** (§7) e o **conjunto inicial de tiers** (§HIP-007 §4).
- Confirmar **chave de idempotência** e **chave de equivalência** (§3, §6).
- Definir **cadência de background** realista por plataforma (§3).
- Só então: **Etapa 4 — Implementação** (ordem definida na aprovação).